

UNIVERSIDAD POLITÉCNICA DE CHIAPAS
MAESTRÍA EN CIENCIAS E INNOVACIÓN TECNOLÓGICA

ASIGNATURA
LÓGICA Y PROGRAMACIÓN

EVALUACIÓN 04

TRABAJO GRUPAL

DOCENTE
DR. HORACIO IRÁN SOLÍS CISNEROS

Tuxtla Gutiérrez, Chiapas
05 de marzo de 2026

1. Introducción

1.1. Contexto

El crecimiento del tráfico vehicular en entornos urbanos representa uno de los principales desafíos para la movilidad en ciudades modernas. En el marco del concepto de *ciudades inteligentes*, los sistemas de control de tráfico buscan mejorar la eficiencia del flujo vehicular mediante el uso de tecnologías de información, sensores y algoritmos de optimización que permitan ajustar dinámicamente los tiempos de los semáforos.

En muchas implementaciones tradicionales, la sincronización de semáforos se realiza mediante algoritmos secuenciales que procesan cada intersección de manera independiente y en orden. Este enfoque simplifica la implementación del sistema, pero introduce limitaciones cuando el número de intersecciones aumenta o cuando se requiere procesar información en tiempo cercano al tiempo real.

En este trabajo se analiza inicialmente el comportamiento de un sistema de sincronización de semáforos bajo un modelo secuencial, denominado **Estado P**, con el objetivo de identificar formalmente sus limitaciones computacionales y los cuellos de botella que afectan su rendimiento.

A partir de este análisis, el trabajo plantea posteriormente la construcción de un modelo alternativo, denominado **Estado Q**, en el cual el procesamiento de intersecciones puede realizarse mediante mecanismos de concurrencia o paralelización. La comparación entre ambos estados permitirá evaluar el impacto de la concurrencia en el rendimiento y escalabilidad del sistema de sincronización de semáforos.

1.2. Problema

Considérese una red urbana compuesta por un conjunto de intersecciones controladas por semáforos:

$$I = \{i_1, i_2, \dots, i_n\}$$

Cada intersección requiere ejecutar una serie de operaciones para determinar el estado del semáforo en cada ciclo de control.

- adquisición de datos de tráfico
- cálculo de densidad vehicular
- determinación del tiempo óptimo del semáforo

- actualización del estado del semáforo

Formalmente, para cada intersección i se define la función de procesamiento:

$$F(i) = read(i) \rightarrow compute(i) \rightarrow update(i)$$

1.3. Estado P: Sistema Secuencial

En el estado inicial del sistema, denominado P , el procesamiento de todas las intersecciones se realiza de forma estrictamente secuencial.

$$P = \sum_{i=1}^n F(i)$$

donde:

$$F(i) = read(i) + compute(i) + update(i)$$

El tiempo total de ejecución puede aproximarse como:

$$T_P = \sum_{i=1}^n (t_r + t_c + t_u)$$

donde

- t_r tiempo de lectura
- t_c tiempo de cómputo
- t_u tiempo de actualización

Por lo tanto,

$$T_P = n(t_r + t_c + t_u)$$

lo que implica una complejidad temporal:

$$T_P = O(n)$$

1.4. Hipótesis de Complejidad

El análisis de complejidad permite estimar el comportamiento del sistema conforme aumenta el número de intersecciones en la red de tráfico.

1.4.1. Complejidad por intersección

Cada intersección ejecuta el pipeline:

$$F(i) = read(i); compute(i); update(i)$$

Cada operación tiene un costo constante, por lo que:

$$F(i) = O(1)$$

1.4.2. Complejidad del sistema completo

El sistema procesa n intersecciones de forma secuencial:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

Por lo tanto:

$$T_P = \sum_{i=1}^n F(i)$$

lo cual produce:

$$T_P = O(n)$$

1.4.3. Hipótesis

Se plantea la siguiente hipótesis de complejidad:

Hipótesis:

Si el sistema de sincronización de semáforos se ejecuta de forma secuencial, entonces el tiempo total de ejecución crecerá linealmente con el número de intersecciones procesadas.

Esto implica que, en redes de tráfico grandes, el sistema puede presentar limitaciones de escalabilidad y tiempos de respuesta elevados.

1.5. Cuello de botella

El principal cuello de botella del sistema secuencial radica en que cada intersección debe esperar a que la anterior complete su procesamiento.

Esto implica:

- ausencia de paralelismo
- bloqueo del flujo de procesamiento
- crecimiento lineal del tiempo de ejecución

1.6. Pregunta de investigación

A partir de este análisis surge la siguiente pregunta:

¿Cómo puede optimizarse el procesamiento del sistema para reducir los tiempos de ejecución del modelo secuencial sin comprometer su corrección funcional?

1.7. Especificación formal mediante tripletas de Hoare

$$\{traffic_data(i)\} \text{ read}(i) \{raw_data(i)\}$$

$$\{raw_data(i)\} \text{ compute}(i) \{timing(i)\}$$

$$\{timing(i)\} \text{ update}(i) \{light_state(i)\}$$

El pipeline completo se expresa como:

$$\{traffic_data(i)\} F(i) \{light_state(i)\}$$

y el sistema completo:

$$\{TrafficNetwork\} P \{UpdatedTrafficLights\}$$

1.8. Diagrama del pipeline (composición secuencial)

Estado P : $P = F(i_1); F(i_2); \dots; F(i_n)$

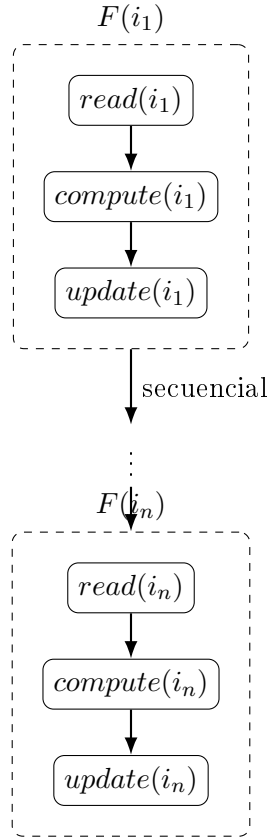


Figura 1: Pipeline secuencial del sistema.